SAVRA

Full-Stack Engineering Assignment

⏰ **Deadline: Saturday, 3:00 PM IST • You have 32 hours only from receipt**
📌 **Submissions will ONLY be considered via the form attached in this email.**

We want your wildest, most well-reasoned implementation — not just a safe answer.

01 | About Savra

Savra is an AI-first EdTech platform built to reduce workload for teachers while improving learning outcomes for students. We operate across three user roles:

Role	Who They Are	What They Do on Savra
👩‍🏫 **Teacher**	Educators & faculty	Create lesson plans, generate quizzes, build presentations, track student progress
🎓 **Student**	School students	Attempt AI-generated quizzes, get instant feedback, track learning progress
🏫 **Admin**	School leadership	Monitor school-wide performance, teacher engagement, and student dashboards

📈 **Scale Context:** We are currently at **2,400 active users** and are targeting **10,000 users by next month.** Our infrastructure and backend need to be production-ready, cost-efficient, and scalable before that happens.

02 | The Problem: PPT Generation is Breaking Us

Current Flow

PPT (Presentation) generation is our most-used feature — teachers generate 100+ presentations per day on Savra. Here is how it currently works:

1	Teacher fills a form — topic, grade, subject, number of slides
2	Request hits our backend synchronously — the teacher waits on screen
3	Backend calls an LLM which generates the text content for each slide
4	Content is injected into pre-built SVG/HTML vector slide templates (AI fills text and data — does NOT design from scratch)
5	Final output is delivered as a .PPTX file with an option to download as PDF

The Pain Points

Problem	Current State	Impact
💸 **Cost per PPT**	₹15 per generation	₹45,000/month at 100/day — unsustainable at 10K users
🔴 **Gemini 3.1 Pro - 503 Errors**	Primary LLM fails under load	Teachers get errors mid-workflow
📉 **Fallback 2.5 Flash Quality Drop**	Fallback model gives worse output	Noticeable quality degradation
🐌 **Generation Time Spike**	30s → 2+ minutes on fallback	Teacher stuck waiting on screen
📈 **Scaling Math**	Volume could 4x at 10K users	Current system will break

🎯 The Core Ask

Redesign the PPT generation system to be cheaper, faster, more reliable, and ready for 10x scale without compromising quality.

This is not a theoretical exercise. Whatever you build or design here is the direction we are likely to move in.

03 | What You Need to Build

Part 1 — Architecture Design Document (Required)

Write a clear technical design document covering the following four areas:

A. System Design

- Proposed architecture diagram (Excalidraw, draw.io, or hand-drawn is fine)
- How requests flow: teacher → backend → LLM → output
- Where queuing, caching, or async processing fits in
- How you handle failures, retries, and fallback without degrading quality

B. Cost Reduction Strategy

- Where are the biggest cost levers in the current system?
- How do you reduce per-PPT cost without switching to a low-quality model?
- Think: caching, prompt optimization, template pre-computation, model routing

C. Reliability Plan

- How do you handle LLM 503s gracefully?
- What does a smart fallback strategy look like — not just 'use a cheaper model'?

D. Scaling Plan

- How does your system behave at 500 PPTs/day? At 2,000?
- What infrastructure decisions need to be made now vs. later?
- Where are the bottlenecks and how do you eliminate them?

Part 2 — Working Prototype (Required)

Build a minimal but functional prototype that demonstrates your core ideas.

Minimum viable prototype must include:

- ☐ A simple UI where a user can submit a PPT generation request (topic, grade, no. of slides)
- ☐ A backend API that accepts the request and processes it

- ☐ Integration with any LLM of your choice (OpenAI, Anthropic, Gemini, or open-source)
- ☐ At least one meaningful optimization implemented — not just described
- ☐ A working output — even a JSON response, HTML slide preview, or basic PPTX

Optimization Ideas (Pick At Least One)

- Async job queue with status polling — teacher doesn't wait, gets notified
- Prompt caching or response caching for similar requests
- Smart model routing — use the expensive model only when truly needed
- Template-first generation — generate content separately from layout injection
- Rate limiting and request deduplication

You don't need to build everything. One thing done well beats five things done poorly.

04 | Recommended Tech Stack

You are free to use any stack. If you choose a different one, briefly justify it in your documentation.

Layer	Recommended Options
Frontend	Next.js / React + Tailwind CSS
Backend	Node.js (Express / Fastify) OR Python (FastAPI)
Job Queue	BullMQ (Redis-backed) OR Celery (Python) OR simple polling
Database	PostgreSQL (primary) + Redis (cache / queue)
LLM Integration	Anthropic SDK / OpenAI SDK / Vercel AI SDK
File Generation	pptxgenjs (Node) OR python-pptx (Python)
Hosting	Railway / Render / Vercel + Railway — free tier is fine
Diagramming	Excalidraw / draw.io / Mermaid for architecture docs

05 | What to Submit

📌 **Important: Submissions will ONLY be considered via the form attached in this email.** Any submission sent directly by email or any other channel will not be reviewed.

Submit everything via a single GitHub repository with the following structure:

```
├── architecture/
    design-doc.md   ← Your architecture document (Part 1)
    diagram.png     ← System architecture diagram
├── backend/
   ← Your API / server code
├── frontend/
   ← Your UI code
└── DECISIONS.md
   ← Key decisions you made and why
```

Your README must also include:

- What you built and what you skipped — and why
- Any assumptions you made about Savra's system

06 | How We Will Evaluate You

We are not looking for perfect code. We are looking for how you think.

Criteria	Weight	What We Look At
🏗️ System Thinking	35%	Does the architecture make sense? Did you identify the right problems?
💰 Cost & Reliability	25%	Are your solutions practical and grounded in real trade-offs?
💻 Code Quality	20%	Is the prototype clean, readable, and does it actually run?
📈 Scalability Awareness	15%	Do you think in load, bottlenecks, and failure modes?

Criteria	Weight	What We Look At
✍️ Communication	5%	Is your doc clear? Can you explain your choices simply? Or Just LLM generated

07 | Pro Tips

Read these carefully — they directly influence how we score you.

1. **Show your thinking, not just your output.** A DECISIONS.md that explains why you made choices is worth more than polished code with no reasoning.
2. **Don't try to boil the ocean.** We've given you a real production problem. Pick the highest-leverage angle and go deep on it. Breadth without depth won't impress us.
3. **The async vs. sync decision is the most important one.** Think hard about what the teacher's experience looks like if the system is async. How do you keep them in the loop?
4. **Cost math matters.** Show us the numbers. If your solution reduces cost by X%, prove it with rough calculations.
5. **Failure modes matter more than the happy path.** We care far more about how your system behaves when things go wrong than when everything works perfectly.
6. **Don't fake the demo.** A simpler system that actually runs is infinitely better than a complex one with mocked responses everywhere.

08 | Bonus Challenge

Optional — Only attempt this if you have completed everything above. This is for the 1%.

Implement a Smart Semantic Caching Layer that:

- Identifies semantically similar PPT requests — e.g., 'Class 8 Photosynthesis 10 slides' and 'Grade 8 Photosynthesis presentation' should hit the same cache
- Uses embeddings to compute similarity (any embedding model is fine)
- Returns cached output if similarity exceeds a defined threshold
- Logs cache hits vs. misses with estimated cost saved per request

Bonus Question to Answer in Your Doc:

"At 10,000 users with 50% being teachers who each generate 2 PPTs/week — what is your projected monthly LLM cost under your new architecture vs. the current system? Show your assumptions."

Good luck. We're rooting for you. 🚀 Submission only via form in mail